
1. الفكرة العامة (Overview)

تطبيق سطح مكتب (Desktop App) مبني بإطار **Electron**، يستقبل من المستخدم نصًا حرًا (بالعربية أو أي لغة) يصف فيه **مشكلة** أو استفسارًا متعلقًا بالسيارات **(عطل، جزء معين، نوع سيارة...)**. يقوم التطبيق بتحليل هذا النص، واستخراج **كلمات مفتاحية** قوية باللغة الإنجليزية خاصة بمجال السيارات وأجزائها، ثم يستخدم هذه الكلمات المفتاحية للبحث في **Pexels API** عن صور وفيديوهات توضيحية، وفي نفس الوقت يستخدم نموذج **YOLO** للكشف عن أجزاء السيارة داخل الصور المسترجعة وتحديدها بمربعات (Bounding Boxes) مع تسميتها.

الهدف النهائي: مستخدم يكتب "فرامل السيارة تصدر صوتًا عند الضغط عليها" → التطبيق يشخص المشكلة، يعطي شرحًا دقيقًا، يجلب صورًا/فيديوهات حقيقية لأجزاء الفرامل من Pexels، ويكشف بصريًا موقع الجزء المعني (نظام الفرامل) داخل الصورة.

2. المكونات التقنية الأساسية (Tech Stack)

الطبقة	التقنية	الدور
الواجهة (Frontend/Shell)	**Electron.js** (Node.js + Chromium)	تطبيق سطح مكتب متعدد المنصات (Windows/Linux/macOS)
العقل المدبر (Backend)	**Python**	كل المنطق الذكي: تحليل النص، الترجمة، استخراج الكلمات المفتاحية، الاتصال بـ Pexels، تشغيل YOLO
الجسر بين Electron و REST API	Python محلي (FastAPI/Flask) أو **IPC** / **python-shell** عبر **subprocess**	تواصل الواجهة مع الخادم البايثوني المحلي
الكشف عن أجزاء السيارة	**YOLOv8** (Ultralytics)	تحديد أجزاء السيارة داخل الصور المسترجعة (نموذج مدرب أو مُدرب خصيصًا لاحقًا Custom Dataset)
مصدر الوسائط	**Pexels API** (صور + فيديوهات)	جلب محتوى بصري حقيقي مرتبط بالكلمات المفتاحية
الترجمة/الفهم اللغوي	**deep-translator / googletrans** أو نموذج NLP	تحويل نص المستخدم (عربي) إلى استعلام إنجليزي دقيق
استخراج الكلمات المفتاحية	**spaCy / KeyBERT** / قاموس مجالي مخصص (Domain Dictionary)	استخراج المصطلحات التقنية الدقيقة لأجزاء السيارات
قاعدة بيانات محلية	**SQLite / JSON**	تخزين تاريخ البحث، الكاش، إعدادات المستخدم، مفتاح API
إدارة الحزم بايثون	**pip + virtualenv / PyInstaller**	تجهيز بيئة بايثون قابلة للتوزيع مع التطبيق

3. سير العمل التفصيلي (Workflow)

الخطوة 1 — إدخال النص

- شاشة رئيسية بحقل نص كبير (Text Input) في مقدمة التطبيق.
- المستخدم يكتب وصفًا حرًا لمشكلة أو سؤال حول السيارة (بأي لغة، أساسًا العربية).
- زر "تحليل / بحث" (Analyze).

الخطوة 2 — الفهم والتحليل (NLP Pipeline)

1. ****كشف اللغة**** تلقائيًا.
2. ****الترجمة**** إلى الإنجليزية إذا لزم الأمر (لأن Pexels يعمل بشكل أفضل بمصطلحات إنجليزية).
3. ****استخراج الكيانات المجالية (Domain Entities)****: نوع السيارة، الجزء المذكور (فرامل، محرك، بطارية، إطارات، ناقل الحركة...)، نوع العطل.
4. ****بناء قاموس كلمات مفتاحية قوي**** (Keyword Bank) خاص بمجال السيارات بالإنجليزية، يُستخدم كمرجع لتحسين دقة الاستعلام، مثل:
`brake pad, brake disc, engine block, alternator, timing belt, spark plug, radiator, suspension system, exhaust pipe, transmission, clutch, battery terminal, headlight assembly`
5. توليد ****استعلام بحث نهائي محسن (Optimized Search Query)**** لإرساله إلى Pexels.

الخطوة 3 — جلب المحتوى من Pexels

- استدعاء ****Pexels Photos API**** و ****Pexels Videos API**** بالتوازي.
- تحديد عدد النتائج، الدقة (Resolution)، والتصفية حسب الاتجاه (Orientation).
- تخزين النتائج مؤقتًا (Cache) محليًا لتفادي استهلاك حصة الـ API.

الخطوة 4 — الكشف الذكي عن أجزاء السيارة (YOLO)

- تمرير كل صورة مسترجعة من Pexels إلى نموذج ****YOLOv8****.
- رسم Bounding Boxes حول الأجزاء المكتشفة (عجلة، مصباح، مرآة، هيكل، إلخ) — حسب النموذج المستخدم (COCO العام لا يحتوي أجزاء دقيقة للسيارة، لذا يُنصح لاحقًا بتدريب ****Custom YOLO Dataset**** لأجزاء السيارات الدقيقة).
- عرض النتيجة كصورة معدلة (Annotated Image) مع تسمية كل جزء مكتشف.

الخطوة 5 — عرض النتائج للمستخدم

- شرح نصي دقيق (تشخيص العطل / معلومات الجزء / نوع السيارة).
- معرض صور Grid Gallery مع الصور الأصلية والصور المشروحة بواسطة YOLO.
- مشغل فيديو مدمج لفيدويوهات Pexels ذات الصلة.
- إمكانية حفظ النتيجة أو تصديرها.

4. المميزات الكاملة المطلوبة (Feature List)

أ) ميزات الذكاء الاصطناعي

- [] محرك تحليل نص ذكي (فهم المشكلة الموصوفة).
- [] قاعدة معرفية لمجال السيارات (أنواع، أعطال شائعة، أجزاء).
- [] مترجم عربي → إنجليزي مدمج ودقيق تقنيًا (مصطلحات ميكانيكية).
- [] مستخرج كلمات مفتاحية (Keyword Extraction) مخصص لمجال السيارات.
- [] نموذج YOLO للكشف البصري على أجزاء السيارة.
- [] إمكانية توسعة لاحقة: تدريب نموذج YOLO مخصص (Custom Training) على صور أجزاء السيارات.

- [] نظام تصنيف الثقة (Confidence Score) لكل جزء مكتشف.

(ب) ميزات الوسائط (Pexels)

- [] بحث صور بجودات متعددة.
- [] بحث فيديوهات قصيرة توضيحية.
- [] تصفية حسب النوع (صورة/فيديو)، الاتجاه، الحجم.
- [] كاش محلي للنتائج لتقليل الطلبات.
- [] إدارة مفتاح API (إدخال آمن، تخزين مشفر محليًا).

(ج) ميزات الواجهة (Electron UI/UX)

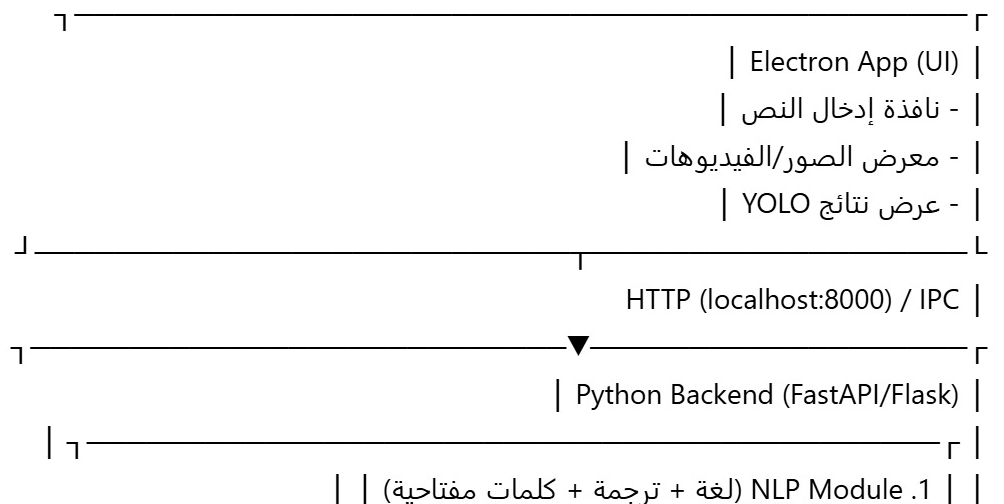
- [] شاشة إدخال نص رئيسية بتصميم بسيط وواضح.
- [] شريط تحميل (Loading State) أثناء التحليل وال جلب.
- [] معرض نتائج (Gallery) قابل للتمرير.
- [] وضعية عرض "قبل/بعد" الكشف (Before/After YOLO Detection).
- [] سجل بحث سابق (History) قابل للاسترجاع.
- [] إعدادات: تغيير مفتاح Pexels، اللغة، جودة الصور، تفعيل/تعطيل YOLO.
- [] دعم الوضع الليلي/النهارى (Dark/Light Mode).
- [] دعم كامل للغة العربية RTL في الواجهة.

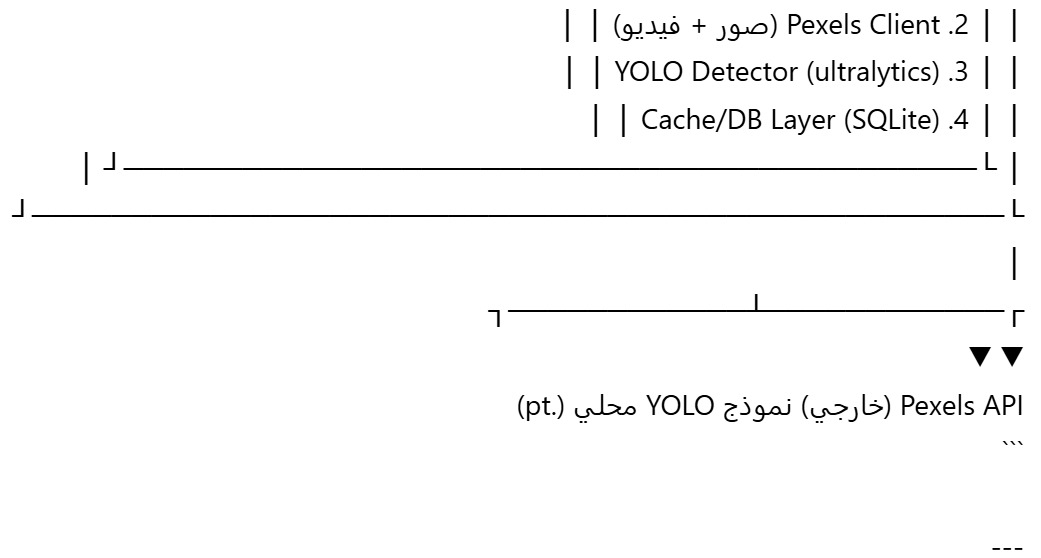
(د) ميزات تقنية/بنوية

- [] خادم Python محلي (FastAPI) يعمل في الخلفية عند تشغيل Electron.
- [] تواصل آمن بين Electron و Python عبر HTTP محلي أو IPC.
- [] معالجة الأخطاء (فشل الاتصال، تجاوز حصة API، نص فارغ...).
- [] نظام Logging لتتبع الأخطاء والأداء.
- [] قابلية التغليف والتوزيع (Packaging) كملف تنفيذي (.exe / .AppImage / .dmg).
- [] بنية معمارية قابلة للتوسعة (إضافة مجالات أخرى غير السيارات مستقبلاً).

5. البنية المعمارية المقترحة (Architecture)

...





6. هيكل المجلدات المقترح (Project Structure)

```
...
/car-diagnostics-app
├── /electron-app # واجهة سطح المكتب
│   ├── main.js # نقطة دخول Electron (تشغيل نافذة + تشغيل خادم بايثون)
│   ├── preload.js
│   ├── package.json
│   └── /src
│       ├── index.html
│       ├── /styles
│       └── /renderer # منطق الواجهة (JS)
├── /python-backend
│   ├── main.py # FastAPI app entry
│   ├── requirements.txt
│   ├── /nlp
│   ├── translator.py
│   ├── keyword_extractor.py
│   └── car_domain_dictionary.json # قاموس مصطلحات السيارات بالإنجليزية
├── /media
├── pexels_client.py # الاتصال بـ Pexels API
├── /vision
├── yolo_detector.py # تحميل نموذج YOLO والتنبؤ
├── /models # ملفات النموذج pt.
├── /storage
├── database.py # SQLite
├── /cache
└── config.py # إدارة مفاتيح API والإعدادات
```

```
└─ /assets
└─ /build # ملفات التغليف (electron-builder)
└─ README.md
...
---
```

7. المكتبات المطلوبة (Python)

```
...
fastapi
uvicorn
requests
pexels-api # أو استدعاء REST مباشرة بدون مكتبة وسيطة
YOLOv8 # ultralytics
opencv-python
deep-translator # أو googletrans
spacy # keybert
sqlite3 (مدمجة في بايثون)
pydantic
python-dotenv # لتخزين مفتاح Pexels بأمان
...
```

8. المكتبات المطلوبة (Node/Electron)

```
...
electron
electron-builder
axios (للتواصل مع خادم بايثون المحلي)
...
```

9. المتطلبات غير الوظيفية (Non-Functional Requirements)

- ****الأمان****: مفتاح Pexels API لا يُخزّن أبدًا كنص صريح داخل الواجهة، بل في ملف إعدادات محلي مشفّر أو `env` على مستوى الباك-إند فقط.
- ****الأداء****: استخدام التخزين المؤقت (Cache) لتفادي تكرار نفس الاستعلامات لـ Pexels وYOLO.
- ****المرونة****: تصميم الكود بشكل Modular بحيث يمكن استبدال مصدر الوسائط (Pexels) أو نموذج الكشف (YOLO) لاحقًا دون إعادة كتابة التطبيق كاملاً.
- ****دعم عدم الاتصال الجزئي****: حفظ آخر النتائج محليًا للعرض دون اتصال بالإنترنت.
- ****قابلية التوسعة****: إمكانية إضافة مجالات أخرى (درجات نارية، شاحنات...) مستقبلاً بنفس البنية.
- ****التوطين (i18n)****: دعم العربية والإنجليزية في الواجهة، مع اتجاه RTL كامل للعربية.

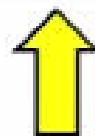
10. مراحل التنفيذ المقترحة (Roadmap)

1. **المرحلة 1**: بناء الواجهة الأساسية Electron + حقل إدخال النص.
2. **المرحلة 2**: بناء خادم Python (FastAPI) والتواصل الأساسي بين الواجهة والخادم.
3. **المرحلة 3**: دمج الترجمة + استخراج الكلمات المفتاحية + قاموس مجال السيارات.
4. **المرحلة 4**: دمج Pexels API (صور + فيديو) وعرض النتائج في الواجهة.
5. **المرحلة 5**: دمج YOLO للكشف عن أجزاء السيارة على الصور المسترجعة.
6. **المرحلة 6**: تحسين الواجهة (Gallery، History، Settings، Dark Mode).
7. **المرحلة 7**: تدريب نموذج YOLO مخصص لأجزاء السيارات (اختياري لتحسين الدقة).
8. **المرحلة 8**: التغليف والتوزيع (Packaging) باستخدام electron-builder.

11. ملاحظة هامة حول Pexels و YOLO

- **Pexels** موقع صور/فيديوهات عامة (Stock Media) وليس مخصصًا للسيارات، لذا دقة النتائج تعتمد بشكل كبير على **جودة الكلمات المفتاحية الإنجليزية** المستخرجة من نص المستخدم — وهذا هو سبب أهمية بناء **قاموس مجالي (Domain Dictionary)** قوي لمصطلحات السيارات.
- **YOLO** بنسخته العامة المدربة على COCO لا يتعرف على أجزاء السيارة الداخلية بدقة** (فرامل، بواجي، حزام التوقيت...)، بل يتعرف فقط على "سيارة" ككائن كامل. للحصول على كشف دقيق لأجزاء السيارة كما هو مطلوب، ستحتاج لاحقًا إلى **تدريب نموذج YOLO مخصص (Custom Dataset)** بصور موسومة لأجزاء السيارات — وهذه نقطة مهمة يجب أخذها بعين الاعتبار عند البناء الفعلي.

* هذه الوثيقة تمثل مواصفات ومتطلبات المشروع فقط، جاهزة للنسخ واستخدامها كمُدخل (Prompt/Spec) لبناء التطبيق فعليًا عبر إحدى منصات الذكاء الاصطناعي البرمجية.*



1